

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д. О. Шевяков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Проектирование приложения интернета
вещей и веб-интерфейса

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Ход выполнения задания.....	4
4 Вывод.....	7
ПРИЛОЖЕНИЕ А	8

1 Цель работы

Целью лабораторной работы является получение практических навыков проектирования IoT-системы и пользовательского интерфейса для отображения состояния устройств. В рамках варианта 1 «Умный дом» необходимо сформировать структуру цифровых двойников, определить связи между элементами, выделить общий абстрактный уровень и реализовать разделение интерфейсов по уровню доступа.

2 Задание

В соответствии с методическими указаниями для лабораторной работы №1 требуется спроектировать IoT-систему, состоящую из 4-5 уникальных классов, определить общие свойства и методы и при необходимости вынести их в абстрактный класс, а также обозначить ассоциативные связи и связи наследования между элементами системы. В работе также необходимо предусмотреть отдельную сущность базы данных, описать состав сохраняемых данных и источники этих данных. Кроме того, требуется разработать не менее двух интерфейсов с различным уровнем доступа, например пользовательский и административный, чтобы показать разделение прав и различия в отображаемой информации.

3 Ход выполнения задания

В ходе выполнения лабораторной работы была спроектирована система «Умный дом», реализованная на TypeScript. В качестве базовой сущности выбран абстрактный класс `Thing`, который задает общую структуру для всех устройств и содержит идентификатор устройства, его имя и признак доступности. Такой подход позволил формализовать единый контракт для цифровых двойников и задать обязательный метод получения статуса устройства. На рисунке 1 изображен модуль базового абстрактного класса, который используется как основа наследования для конкретных устройств.

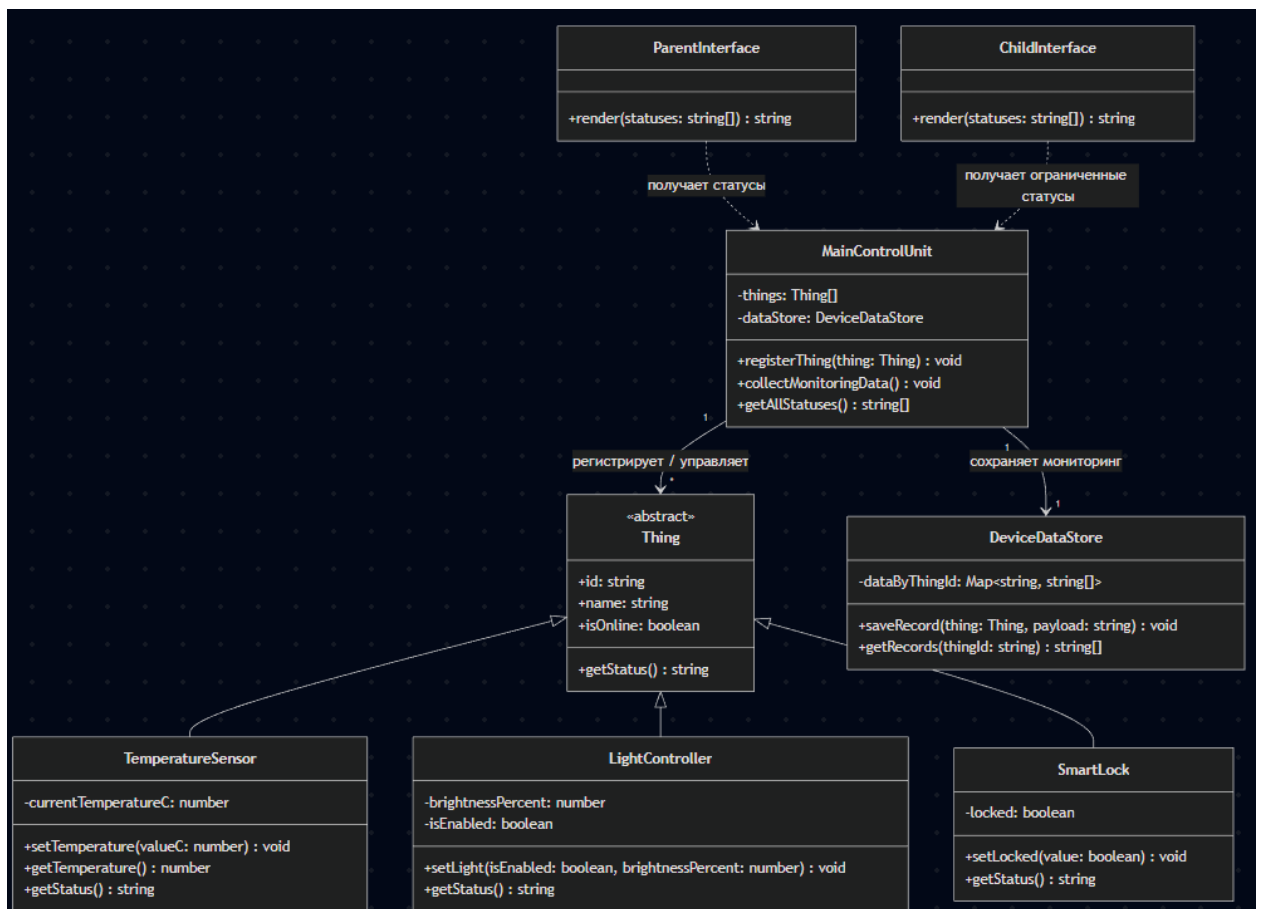


Рисунок 1 - Схема

Структура схемы системы выстроена от общего уровня к частному. В верхней части расположен абстрактный класс `Thing`, от которого наследуются конкретные устройства `TemperatureSensor`, `LightController` и `SmartLock`, что отражает общий набор свойств и единый интерфейс

получения состояния. Центральное место в схеме занимает `'MainControlUnit'`, который связан ассоциацией с множеством устройств и выполняет функции регистрации, координации и запуска сбора мониторинга. С сущностью `'DeviceDataStore'` центральный модуль связан отношением «один к одному», так как именно в это хранилище передаются и сохраняются записи состояния по идентификаторам устройств. На уровне пользовательского взаимодействия выделены `'ParentInterface'` и `'ChildInterface'`, которые используют данные системы с разной степенью доступа: первый интерфейс предоставляет полный просмотр и управление, второй выводит только ограниченный набор безопасных параметров.

Далее были реализованы специализированные классы `'TemperatureSensor'`, `'LightController'` и `'SmartLock'`, которые наследуются от `'Thing'` и описывают поведение конкретных устройств умного дома. Класс температурного датчика хранит текущее значение температуры и формирует строку мониторинга, класс освещения отвечает за состояние включения и уровень яркости, а класс замка хранит состояние «открыт/закрыт». Такое разделение обязанностей позволяет изолировать логику отдельных типов устройств и при этом сохранить единый способ работы с ними через базовый интерфейс абстрактного класса.

Для организации взаимодействия между объектами был добавлен класс `'MainControlUnit'`, выполняющий роль центрального управляющего узла. Он регистрирует устройства, запрашивает у них текущее состояние и инициирует сбор мониторинговых данных. Хранение результатов мониторинга реализовано в классе `'DeviceDataStore'`, который сохраняет историю состояний по идентификатору устройства. Такая структура отражает требование методических указаний о выделении отдельного объекта, отвечающего за координацию и передачу данных, а также о наличии сущности базы данных.

Для выполнения требования по пользовательским интерфейсам были спроектированы два представления: `'ParentInterface'` и `'ChildInterface'`.

Интерфейс родителя отображает полный список статусов и подразумевает полный контроль системы, а интерфейс ребенка показывает только безопасные данные мониторинга, связанные с температурой и освещением, без доступа к критичным элементам, например состоянию дверного замка. В рамках демонстрации реализована функция `demoVariant1SmartHome()`, которая создает объекты системы, настраивает начальные состояния устройств, выполняет сбор данных и выводит два разных представления в консоль.

4 Вывод

В результате выполнения лабораторной работы была спроектирована и реализована модель IoT-системы «Умный дом», соответствующая требованиям варианта 1 и методическим указаниям к первой лабораторной работе. В архитектуре выделен абстрактный базовый класс, реализованы классы устройств с наследованием, добавлен центральный управляющий узел и отдельная сущность хранения данных, а также разработаны два пользовательских интерфейса с различными правами доступа. Полученная структура является логически завершенной для этапа проектирования и может быть использована как основа для дальнейших лабораторных работ, связанных с веб-интерфейсом, обменом данными и управлением устройствами.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
abstract class Thing {  
    constructor(  
        public readonly id: string,  
        public readonly name: string,  
        public isOnline: boolean = true  
    ) {}  
  
    abstract getStatus(): string;  
}  
  
class TemperatureSensor extends Thing {  
    private currentTemperatureC: number;  
  
    constructor(id: string, name: string, initialTemperatureC: number) {  
        super(id, name, true);  
        this.currentTemperatureC = initialTemperatureC;  
    }  
  
    setTemperature(valueC: number): void {  
        this.currentTemperatureC = valueC;  
    }  
  
    getTemperature(): number {  
        return this.currentTemperatureC;  
    }  
}
```

```

getStatus(): string {
    return `${this.name}: ${this.currentTemperatureC.toFixed(1)} °C`;
}
}

```

```

class LightController extends Thing {
    private brightnessPercent = 0;
    private isEnabled = false;

    setLight(isEnabled: boolean, brightnessPercent: number): void {
        this.isEnabled = isEnabled;
        this.brightnessPercent = Math.max(0, Math.min(100, brightnessPercent));
    }
}

```

```

getStatus(): string {
    return `${this.name}:  ${this.isEnabled  ?  "BKJI"   :  "BЫIKJI"},
яркость=${this.brightnessPercent}%`;
}
}

```

```

class SmartLock extends Thing {
    private locked = true;

    setLocked(value: boolean): void {
        this.locked = value;
    }
}

```

```

getStatus(): string {
    return `${this.name}: ${this.locked ? "ЗАКРЫТ" : "ОТКРЫТ"}`;
}

```

```
}
```

```
class DeviceDataStore {
```

```
    private readonly dataByThingId = new Map<string, string[]>();
```

```
    saveRecord(thing: Thing, payload: string): void {
```

```
        const records = this.dataByThingId.get(thing.id) ?? [];
```

```
        records.push(payload);
```

```
        this.dataByThingId.set(thing.id, records);
```

```
    }
```

```
    getRecords(thingId: string): string[] {
```

```
        return this.dataByThingId.get(thingId) ?? [];
```

```
    }
```

```
}
```

```
class MainControlUnit {
```

```
    private readonly things: Thing[] = [];
```

```
    constructor(private readonly dataStore: DeviceDataStore) {}
```

```
    registerThing(thing: Thing): void {
```

```
        this.things.push(thing);
```

```
    }
```

```
    collectMonitoringData(): void {
```

```
        for (const thing of this.things) {
```

```
            // Сохраняем снимок текущего состояния каждого устройства с временной меткой.
```

```
            const line = `[${new Date().toISOString()}] ${thing.getStatus()}`;
```

```

    this.dataStore.saveRecord(thing, line);
  }
}

```

```

getAllStatuses(): string[] {
  return this.things.map((thing) => thing.getStatus());
}
}

```

```

class ParentInterface {
  render(statuses: string[]): string {
    return [
      "Интерфейс родителя (администратора)",
      "Возможности: полный обзор и управление",
      ...statuses
    ].join("\n");
  }
}

```

```

class ChildInterface {
  render(statuses: string[]): string {
    const safeLines = statuses.filter(
      (line) => line.includes("Температурный датчик") || line.includes("Свет")
    );
    return [
      "Интерфейс ребенка (ограниченный)",
      "Возможности: только просмотр температуры и света",
      ...safeLines
    ].join("\n");
  }
}

```

```
}
```

```
function demoVariant1SmartHome(): void {  
  const db = new DeviceDataStore();  
  const mcu = new MainControlUnit(db);  
  
  const temperature = new TemperatureSensor("sensor-1", "Температурный  
датчик", 22.5);  
  const light = new LightController("light-1", "Свет в гостиной");  
  const lock = new SmartLock("lock-1", "Замок входной двери");  
  
  light.setLight(true, 65);  
  lock.setLocked(true);  
  
  mcu.registerThing(temperature);  
  mcu.registerThing(light);  
  mcu.registerThing(lock);  
  mcu.collectMonitoringData();  
  
  const statuses = mcu.getAllStatuses();  
  const parentUi = new ParentInterface();  
  const childUi = new ChildInterface();  
  
  console.log(parentUi.render(statuses));  
  console.log("-----");  
  console.log(childUi.render(statuses));  
  console.log("-----");  
  console.log("Сохраненные записи для sensor-1:", db.getRecords("sensor-1"));  
}
```

```
demoVariant1SmartHome();
```